

Jest do zero

Aula prática de 40 minutos: do Codespace ao primeiro projeto de testes

OBJETIVO DA AULA

Ao final, a turma será capaz de criar, executar e interpretar testes simples com Jest, inclusive um teste isolado.

Mapa dos 40 minutos

0-12

Ambiente

12-25

PASS / FAIL

25-40

Prática

0-4 MIN

1. Abra o Codespace

- Entre no GitHub e abra o repositório preparado para a aula.
- Clique em Code -> Codespaces -> Create codespace on main.
- Espere o editor abrir no navegador. O Codespace já traz um terminal Linux e o VS Code online.

Terminal - confira o ambiente

```
1 $ node --version
2 $ npm --version
3 $ pwd
```

Se os dois primeiros comandos mostrarem versões, o ambiente está pronto.

FALA SUGERIDA

"O Codespace é nosso laboratório. Nada precisa ser instalado no computador da escola."

4-12 MIN

2. Prepare o projeto e instale o Jest

Terminal - projeto aula-jest

```
1 $ mkdir aula-jest
2 $ cd aula-jest
3 $ npm init -y
4 $ npm install --save-dev jest
```

- npm init -y cria o arquivo package.json.
- save-dev registra o Jest como ferramenta usada durante o desenvolvimento.
- No package.json, troque o script de teste por "test": "jest".

CHECAGEM RÁPIDA

Pergunte: qual arquivo guarda as informações do projeto? Resposta esperada: package.json.

Seu primeiro teste

Números, leitura do resultado e execução de testes individuais

12-18 MIN

3. Crie a função e o arquivo de teste

```
soma.js
1 function somar(a, b) {
2   return a + b;
3 }
4
5 module.exports = somar;
```

```
soma.test.js
1 const somar = require('./soma');
2
3 test('2 + 3 deve ser 5', () => {
4   expect(somar(2, 3)).toBe(5);
5 });
```

LEIA O TESTE EM PORTUGUÊS

- `test(...)` cria e descreve o teste
- `expect(...)` executa o que será verificado
- `.toBe(5)` define o resultado exato esperado

18-22 MIN

4. Execute, leia o PASS e provoque um FAIL

```
Terminal - todos os testes
1 $ npm test
2 PASS ./soma.test.js
3 Tests: 1 passed, 1 total
```

DEMONSTRAÇÃO ESSENCIAL

Troque somente `toBe(5)` por `toBe(10)`. Rode de novo e destaque: Expected: 10 e Received: 5.

Mensagem-chave: um teste que falha não é inútil. Ele está avisando que o resultado real é diferente do combinado.

22-25 MIN

5. Rode somente o que você quer investigar

TUDO

`$ npm test`

UM ARQUIVO

`$ npx jest soma.test.js`

UM TESTE

`$ npx jest -t "2 + 3"`

PERGUNTA À TURMA

"Quando vale rodar só um teste?" Quando quero corrigir um problema sem executar toda a suíte.

Prática: dois novos usos

Além de contas: teste de texto e teste de verdadeiro/falso

25-27 MIN

6. Prepare duas funções simples

```
utilidades.js

1 function saudacao(nome) {
2   return 'Olá, ' + nome + '!';
3 }
4 function maiorDeIdade(idade) {
5   return idade >= 18;
6 }
7 module.exports = { saudacao, maiorDeIdade };
```

USO 1 | TEXTO

Jest também compara palavras e frases. A escrita precisa ser exatamente igual, inclusive pontuação.

USO 2 | VERDADEIRO / FALSO

Uma regra pode responder apenas sim ou não. Em JavaScript, isso aparece como true ou false.

```
utilidades.test.js

1 const { saudacao, maiorDeIdade } =
2   require('./utilidades');
3
4 test('cumprimenta Ana', () => {
5   expect(saudacao('Ana'))
6     .toBe('Olá, Ana!');
7 });
```

```
continue em utilidades.test.js

1 test('18 é maior de idade', () => {
2   expect(maiorDeIdade(18))
3     .toBe(true);
4 });
5 test('17 não é maior', () => {
6   expect(maiorDeIdade(17))
7     .toBe(false);
8 });
```

27-36 MIN

7. Duplas: escreva, execute, corrija

1 Digite

Crie os dois arquivos exatamente como nos exemplos.

2 Execute

Use `npx jest utilidades.test.js`.

3 Faça falhar

Troque Ana por Bia só no resultado esperado.

4 Explique

Diga ao colega o que Expected e Received mostram.

36-40 min | Fechamento em três perguntas

- 1 Para que serve o Jest?
- 2 O que significa expect?
- 3 Como rodo só um teste?

Para automatizar verificações no código.

O valor real que quero conferir.

`npx jest -t "parte do nome"`

FRASE FINAL

"Testar é combinar antes o resultado esperado e pedir ao computador que avise quando algo sair diferente."